

- **Let's start with a real-life example. Let's say we have 10 mobile phones and 2 chargers, if we add them it will be 10 mobiles and 2 chargers it will not become 12 mobiles or 12 chargers because they both are different objects. Just like that in C language, different data types are different objects, so they cannot be added or any arithmetic operation is not possible until and unless we convert them to the same data type.**

Type Conversion

- convert the value of one data type to another type. This is known as type conversion.
- example:

```
int x = 5;
```

```
int y = 2;
```

```
int sum = 5 / 2;
```

```
printf("%d", sum); // Outputs 2
```

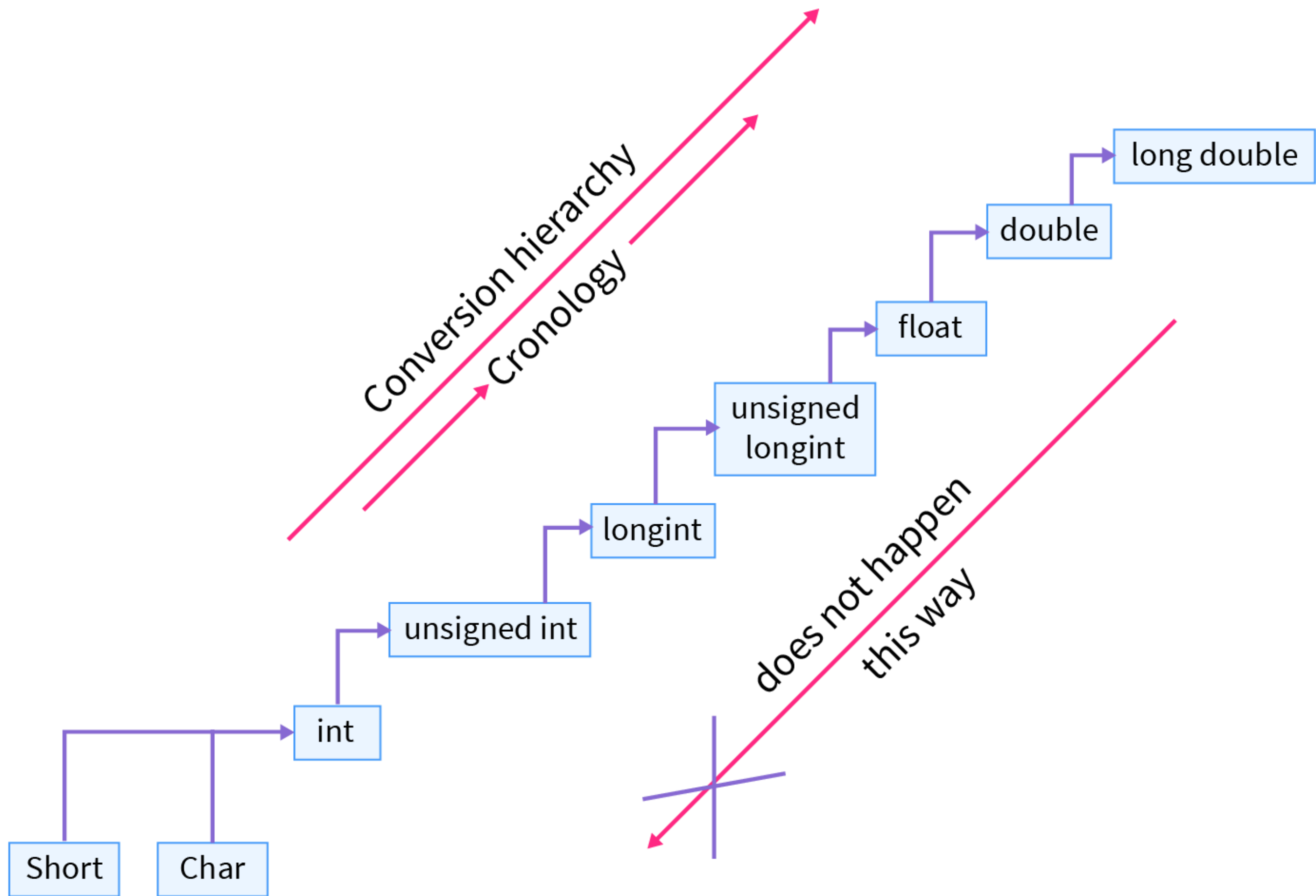
- We use it to convert the data type of any variable without losing the actual meaning that it holds.

Rules:

- **If we are performing a conversion on two of the different data types in a program, then the conversion of the lower data type to the higher data type will occur automatically.**
- **If we suppose that we are performing the type conversion operation among two different data types like float and int, then the resultant value would be the floating data type (float).**

```
int x = 4;  
float a = 12.4, b;  
b = a / x;
```

- Above example, variable x will get converted to the float data type (float) automatically, and it is comparatively a bigger data type in C. Here, the variable x and the variable a would be equal in terms of their data types. Thus, the value of b would be equal to $12.4/4.0=3.1$.



```
#include <stdio.h>
int main() {
int val = 21;
char xyz = 'T';
float total;
total = val + xyz;
printf("Let us look at the case of Arithmetic Conversion Hierarchy with Type
Casting\n");
printf("The total of all these values is equivalent to: %f\n", total);
}
```

output ; 105.000000

Types of type conversion

- **Implicit type conversion.**
- **Explicit type conversion.**

Implicit type conversion

- The implicit type conversion takes place when more than one data type is present in an expression.
- It is done by the compiler itself it is also called automatic type conversion.
- Here the automatic type conversion takes place in order to prevent data loss, as the datatypes are upgraded to the variable with datatype having the largest value.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int a,b = 2;
    float x = 9.5;
    double y = 10.5;
    long int z = 50;
    double d;
    a = z/b+b*x-y;
    printf("When stored as integer : %d\n",a);
    d = z/b+b*x-y;
    printf("When stored as double : %f\n",d);
    return 0;
}
```

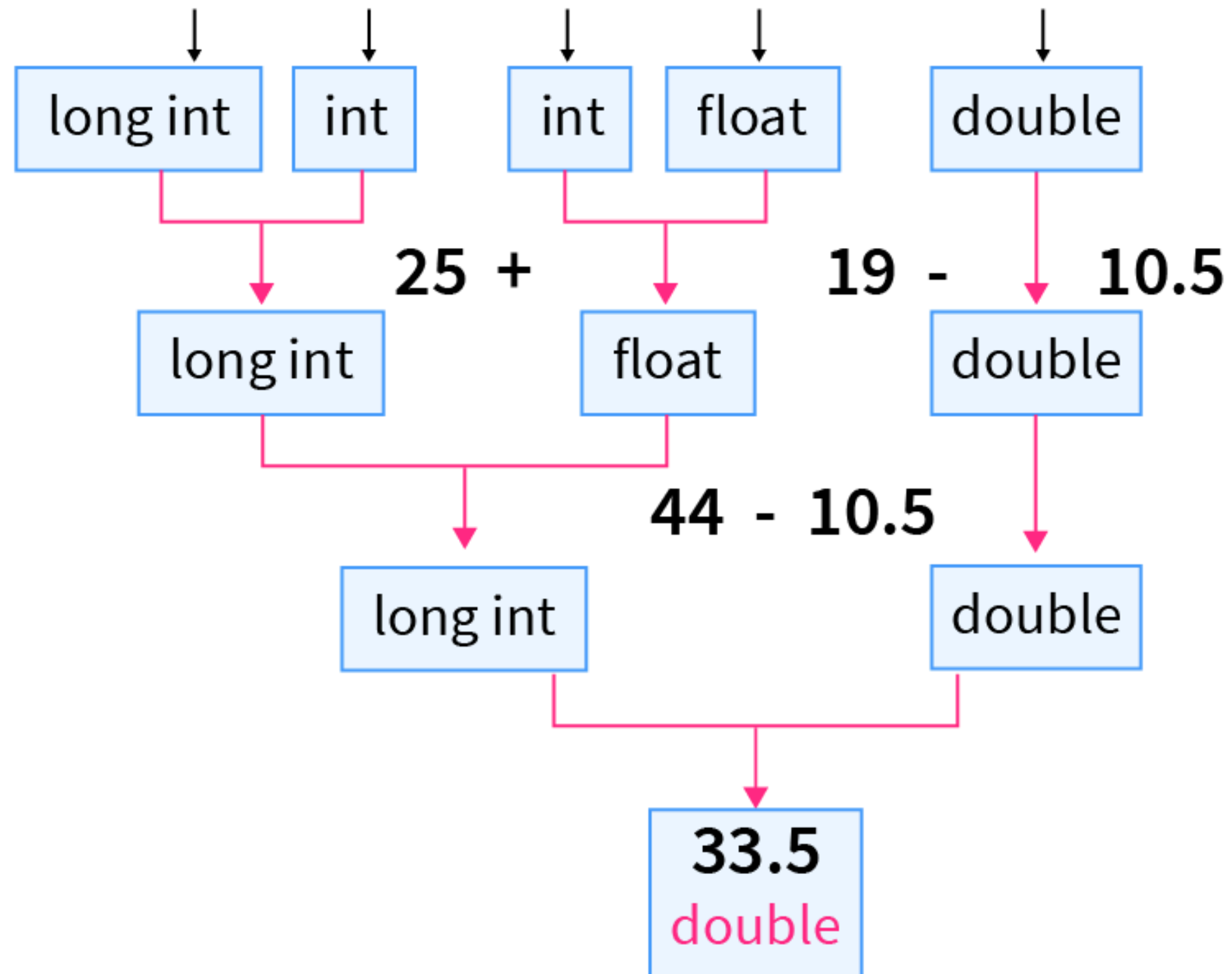
Output:

When stored as integer : 33

When stored as double : 33.500000

a = z / b + b * x - y i

a = 50 / 2 + 2 * 9.5 - 10.5



Explicit Type Conversion

- Explicit conversion is done manually by placing the type in parentheses () in front of the value.

- // Manual conversion: **code 1**

int to float

```
float sum = (float) 5 / 2;
```

```
printf("%f", sum); // 2.500000
```

code 2 ;

```
int num1 = 5;
```

```
int num2 = 2;
```

```
float sum = (float) num1 / num2;
```

```
printf("%f", sum); // 2.500000
```

- **The explicit type conversion is also called type casting in other languages.**
- **It is done by the programmer, unlike implicit type conversion which is done by the compiler.**

Syntax for explicit conversion :

(datatype) expression

```
#include <stdio.h>
int main()
{
    int a = 3;
    int b = 20;
```

```
float x = b/a; //without explicit type conversion
printf("Without explicit type conversion: %f\n",x);
```

```
x = b/(float)a; //with explicit type conversion
printf("With explicit type conversion: %f\n",x);
```

```
    return 0;
}
```

Output:

Without explicit type conversion:

6.000000

With explicit type conversion:

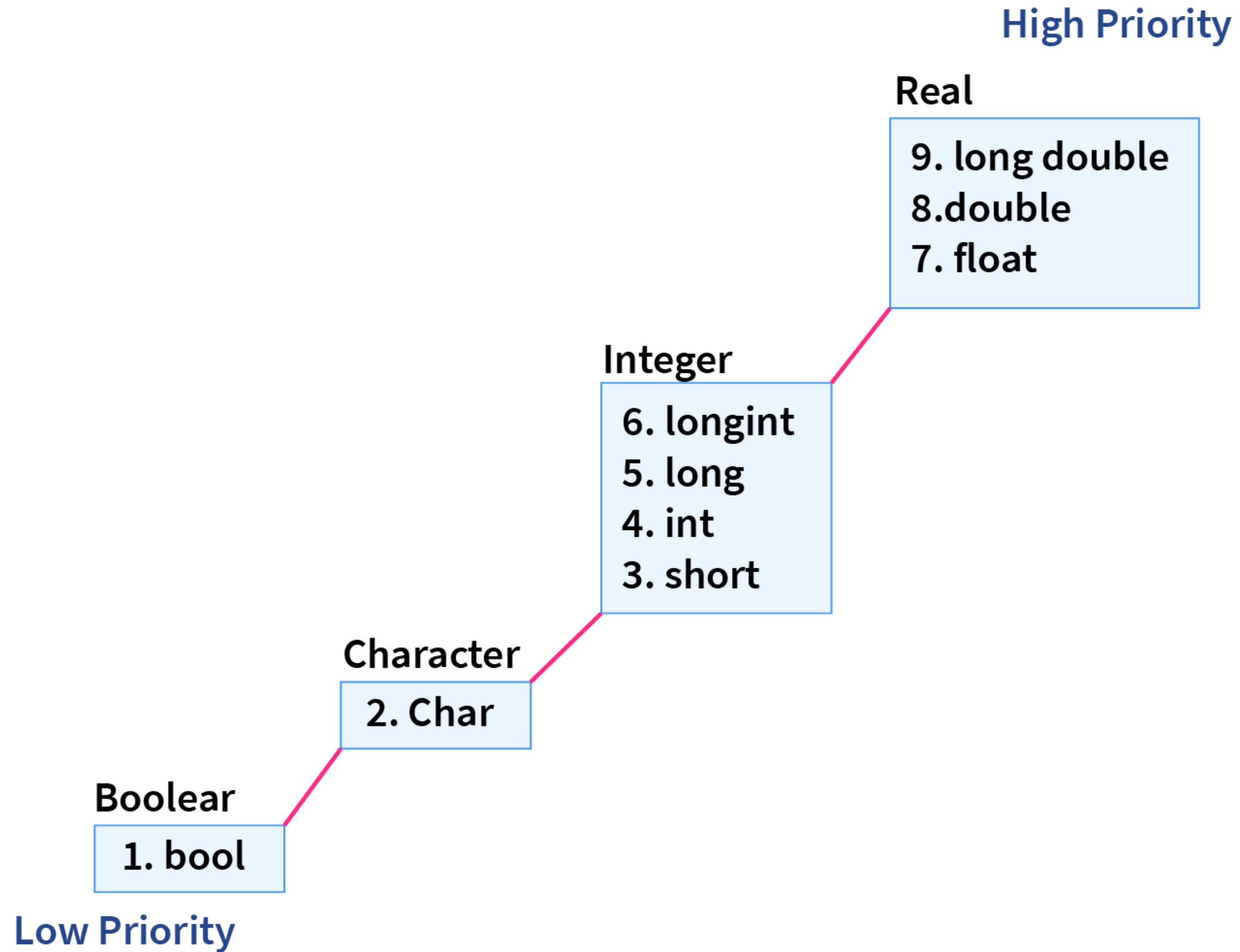
6.666667

- there are a few points that should be noted.
- The datatype conversion of an (int) into (float) is temporary, the integer variable will be treated as int if we use it in any other expression.
- The value of $x = b/(\text{float})a$ is not equal to $x = \text{float}(b/a)$ because in the second case the arithmetic operation happens before datatype conversion whose output is 6.000 and it will be assigned to x as a float with the value 6.000 and result will remain same.

Occurrences of Implicit Type Conversion in C.

- Conversion Rank
- Conversions in Assignment Expressions
- Promotion
- Demotion
- Conversions in other Binary Expressions

Conversion Rank



```
#include <stdio.h>
#include <stdbool.h>
```

```
int main() {
    char a = 'S';
    bool b = true;
    int c = 2;

    printf("bool+char=char :%c\n", b + a);

    return 0;
}
```

Here the variable 'a' is declared as char, while the variable 'b' is declared as boolean, which has a lower conversion rank than char. When we add the two, we get char and the result becomes ('S' + 1), which is T. So the output we get is T.

Output:
bool+char=char :T

Conversions in Assignment Expressions

- starting with the assignment operator = there are two sides to it first is (left) of '=' and (right).
- So the occurrence of operands has the following conditions.

If the operands are different, then with respect to the conversion rank the right-side operand data type will be converted to the left-hand side operand datatype.

Eg.

int a = 'x';

Here the operand on the left side has a higher rank, so the right-hand operand will be converted to left-hand type i.e. integer and it will take the ASCII value of x i.e. 120.

- **f the right operand has a higher rank than the left side, then it will be demoted to the operand data type of the left side**

Eg.

```
int b = 112.3;
```

Here the right operand is int and left is float so the value 112.3 will be converted to 112 as an integer.

```
#include <stdio.h>
```

```
int main() {
```

```
int b = 'a';
```

```
int k = 112.3;
```

```
printf("value of b : %d\n", b); //char is converted to int
```

```
printf("value of k : %d\n", k); //float is converted to int
```

```
return 0;
```

```
}
```

Output:

value of b: 97

value of k: 112

- Here, the char a is converted into int and the ASCII value of a which is 97, is assigned to int a. The 112.3 value is converted to int 112, where the digits after the decimals are excluded.

Promotion in data type conversion

- In promotion, the datatype of lower rank is converted into a datatype of higher. If we take the example of char,int, and float so char is a subset of int and int is a subset to float so if we convert accordingly then the promotion generally does not create any problems and it is done to avoid data loss.

```
#include <stdio.h>
int main() {
    int a = 'x';
    printf("%d", a);
    return 0;
}
```

Output:
120

Demotion

- In demotion when the data type of the higher rank is converted into the data type of the lower rank. If we take the example of char, int and float the float is a superset of int and int is a superset of char. So demotion can create problems and give unpredictable results.
- Let's take an example of char and int. when a char is converted to int it takes the value of char as ASCII value but the ASCII values are limited to 256 if we demote int 257 to char, it will overflow and the compiler will give warnings.

```
#include <stdio.h>
```

```
int main() {
```

```
    char x2 = 257;
```

```
    printf("%c\n", x2);
```

```
    return 0;
```

```
}
```

**main.c:13:11: warning: overflow in conversion from ‘int’ to ‘char’ changes value from
‘257’ to ‘1’ [-Woverflow]**

13 | char x2 = 257;

| ^~~

- **Important Points about Implicit Conversions**

- If one operand is of higher rank and the other operand is of the same rank or lower rank then the output of the operation will be that higher rank. (Refer to the rules in the above tables for conversion). It is also called promotion of type.
- When the datatype double is converted into float then the digits are rounded off.
- When any arithmetic operation between int and char happens then the char is converted to its ASCII value and then the operation is performed.
- When the int type is converted into float type or the float type is converted to double type then there is no increase in the accuracy.

Conclusion.

- The implicit type conversion is also called automatic type conversion as it is automatically done by a compiler.
- The type conversion which is done by the programmer himself is called explicit type conversion.
- There is conversion rank according to which the implicit type conversion happens in C.
- There are many occurrences of implicit type conversion like in Assignment Expressions, Binary Expressions, etc.

```
#include <stdio.h>
int main(){
int digit = 21;
char a = 'T';
int total;
printf("Let us look at Integer Promotion in C\n");
total = digit + a;
printf("The total of all the digits is equal to: %d\n", total);
return 0;
}
```

The output :

Let us look at Integer Promotion in C

The total of all the digits is equal to: 105

C Language In-built Data Type Casting Functions

There are basically 5 different in-built type casting functions available in the C language. These are:

atof(): We use it to convert the data type string into the data type float.

atoi(): We use it to convert the data type string into the data type int.

atbol(): We use it to convert the data type string into the data type long.

itoba(): We use it to convert the data type int into the data type string.

ltoa(): We use it to convert the data type long into the data type string.

The Differences Between Type Conversion and Type Casting in C

- Though both of these terms are often used interchangeably – there is a significant difference between the process of type conversion and type casting. Here is how:
- In case the conversion of one data type to another one occurs **without any need for human interaction, it is referred to as type conversion**. On the other hand, **whenever a user performs the conversion of any data type to another one, it is referred to as type casting in C**.
- **One has to make use of the '()' operator in the case of type casting, but there is no need for any operator in the case of type conversion in C language**.
- **The process of type casting occurs when a program gets written. On the other hand, we perform type conversion during the process of compilation**.

- what would be the output obtained out of the program mentioned below?

```
#include <stdio.h>
```

```
main() {
```

```
int a = 17;
```

```
char b = 'c';
```

```
int total;
```

```
total = a + b;
```

```
printf("The addition of both the integers would return the value : %d\n", total );
```

```
}
```

The addition of both the integers would return the value : 116

What would be the output obtained out of the program mentioned below?

```
#include <stdio.h>
```

```
main() {
```

```
int p = 17;
```

```
char q = 'A';
```

```
float total;
```

```
total = p + q;
```

```
printf("The total of both the digits is equal to : %f\n", total );
```

```
}
```

The total of both the digits is equal to : 82.000000

Type Checking

- The existence of types is useless without a process of verifying that those types make logical sense in the program so that the program can be executed successfully.
 - This is where type checking comes in.
 - Type checking is the process of verifying and enforcing the constraints of types, and it can occur either at compile time (i.e. statically) or at runtime (i.e. dynamically).
 - Type checking is all about ensuring that the program is type-safe, meaning that the possibility of type errors is kept to a minimum.
 - A type error is an erroneous program behavior in which an operation occurs (or tries to occur) on a particular data type that it's not meant to occur on. This could be a situation where an operation is performed on an integer with the intent that it is a float, or even something such as adding a string and an integer together:
-
- **$x = 1 + "2"$**

Type Checking

- Type checking is the process of verifying that the types of all variables, expressions, and functions are according to the rules of the programming language.
- It means operations operate on compatible data types, which helps in preventing errors and hence ensures correctness of the program.
- Type checking might be performed at compile time, called static type checking, or it could be performed during execution, and this is termed dynamic type checking.
- staticType checking is done during semantic phase of compiler.

- When a program is considered not type-safe, there is no single standard course of action that happens upon reaching a type error.
- Many programming languages throw type errors which halt the runtime or compilation of the program, while other languages have built-in safety features to handle a type error and continue running (allowing developers to exhibit poor type safety).
- Regardless of the aftermath, the process of type checking is a necessity.

Types of Type Checking

- **Static Type Checking.**
- **Dynamic Type Checking.**

Static Type Checking

- A language is statically-typed if the type of a variable is known at compile time instead of at runtime.
- **It means that the type of a variable is known at compile time**
- Common examples of statically-typed languages include Ada, C, C++, C#, JADE, Java, Fortran, Haskell, ML, Pascal, and Scala.
- The big benefit of static type checking is that it allows many type errors to be caught early in the development cycle.
- Static typing usually results in compiled code that executes more quickly because when the compiler knows the exact data types that are in use, it can produce optimized machine code (i.e. faster and/or using less memory).
- Static type checkers evaluate only the type information that can be determined at compile time, but are able to verify that the checked conditions hold for all possible executions of the program, which eliminates the need to repeat type checks every time the program is executed

Java code

```
String a;  
a = "Java is good";
```

- first line declares that the variable “a” is bound to the string type at compile time.
- In the second line, “a” is assigned a value which is not a string object. For example, if we say, a =5, the compiler would raise an error because of incompatible types.

Dynamic Type Checking

- Dynamic type checking is the process of verifying the type safety of a program at runtime.
- type of a variable is allowed to change over its lifetime.
- Common dynamically-typed languages include Groovy, JavaScript, Lisp, Lua, Objective-C, PHP, Prolog, Python, Ruby, Smalltalk and Tcl.
- dynamic type checking typically results in less optimized code than does static type checking; it also includes the possibility of runtime type errors and forces runtime checks to occur for every execution of the program (instead of just at compile-time). However, it opens up the doors for more powerful language features and makes certain other development practices significantly easier.

python dynamic typing

variable a is assigned to a string

```
a="hello"
```

```
print(type(a))
```

variable a is assigned to an integer

```
a = 5
```

```
print(type(a))
```

type of variable “a” is allowed to change and Python correctly infers the type as it changes.

Weakly Typed : Think of weakly typed languages as individuals who are extremely agreeable. When you attempt to perform operations with completely different data types, they readily handle implicit type conversion on your behalf (Doing all sort of things to keep the party going)

Strongly Typed : On the other hand, strongly typed languages are the complete opposite. They do not compromise when it comes to type rules. If you want to add a string with an int, then you have to do it explicitly. (No implicit coercion of types) and you cannot randomly cast any type to any other type, there are rules, strict rules.

strongly-typed language

- in which variables are bound to specific data types, and will result in type errors if types to not match up as expected in the expression – regardless of when type checking occurs.
- in which each type of data, such as integers, characters, hexadecimal and packed decimals, is predefined as part of the programming language, and all constants or variables defined for a given program must be described with one of the data types.
- To give an example, in the following code block repeated from above, a strongly-typed language would result in an explicit type error which ends the program's execution, thus forcing the developer to fix the bug:
 - `x = 1 + "2"`
- We often associate statically-typed languages such as Java and C# as strongly-typed (which they are)
- However, ruby and python (both of which are dynamically-typed) are also strongly-typed languages

- **Statically typed languages focus on resolving type information of all the expressions at compile time,**
- **Strongly typed languages prevents implicit type conversion and enforce strict rules for explicit type casting.**

- **Statically typed but weakly typed (C language)**
- **Dynamically typed but strongly typed (Python)**

Take a look at this example,

```
int add_one(int a){  
    return a + 1;  
}
```

```
int main() {  
    int res1 = add_one('a');  
    int res2 = add_one(97);  
    printf("%d %d", res1, res2); // 98 98  
    return 0;  
}
```

- **E**ven though our function `add_one` is telling explicitly(statically) that it wants a `int` we are still able to pass a `char` ('a') and get our function to work.
- The reason behind this behavior lies in C's weakly typed nature, where it implicitly converts the character 'a' to its ASCII representation of 97.

In a weakly typed language, I can write the below code without getting any errors,

```
int num = 42;
```

```
float pi = 3.14;
```

```
// Implicit conversion: int to float
```

```
num = pi;
```

Consider Java, which is statically and strongly(mostly) typed language, if I try to do the same it will throw error.

```
int num = 42;
```

```
double pi = 3.14;
```

```
// error: incompatible types: possible lossy conversion from double to int
```

```
num = pi;
```

- Dynamically and Strongly typed — Python
- In Dynamically typed languages, there's no type checks done by compiler.
- **The strongly typed nature make sure no implicit type conversion happens behind the scenes and there are strict type rules on every operation.**

Let's see what I mean by that with an example,

```
def add_one(num):  
    return num + 1
```

```
def main():  
    # TypeError: can only concatenate str (not "int") to str  
    res1 = add_one('a')  
  
    res2 = add_one(97)  
    print(res1, res2)
```

- Python being a dynamically typed language doesn't put any restriction on the type of the argument to `add_one` , However, the validity of operations within the function is determined at runtime based on the type of the argument passed.
- For Example, when passing a character as the argument to `add_one`, the arithmetic addition with 1 becomes invalid. The strongly typed nature of python doesn't do implicit conversion like C. If types are not matching, it will throw `TypeError` at runtime.

